JAVA LAB WEEK 13

Title: Collection Framework – Letter Combinations of a Phone Number

1. Objective / Aim

To write and execute a Java program using the Collection Framework to generate all possible letter combinations of a given phone number. The program uses the List interface and ArrayList to store and display all possible combinations for the digits from 2 to 9.

2. System Requirements

Hardware Requirements:

- Computer or laptop
- Minimum 4 GB RAM
- Keyboard and monitor

Software Requirements:

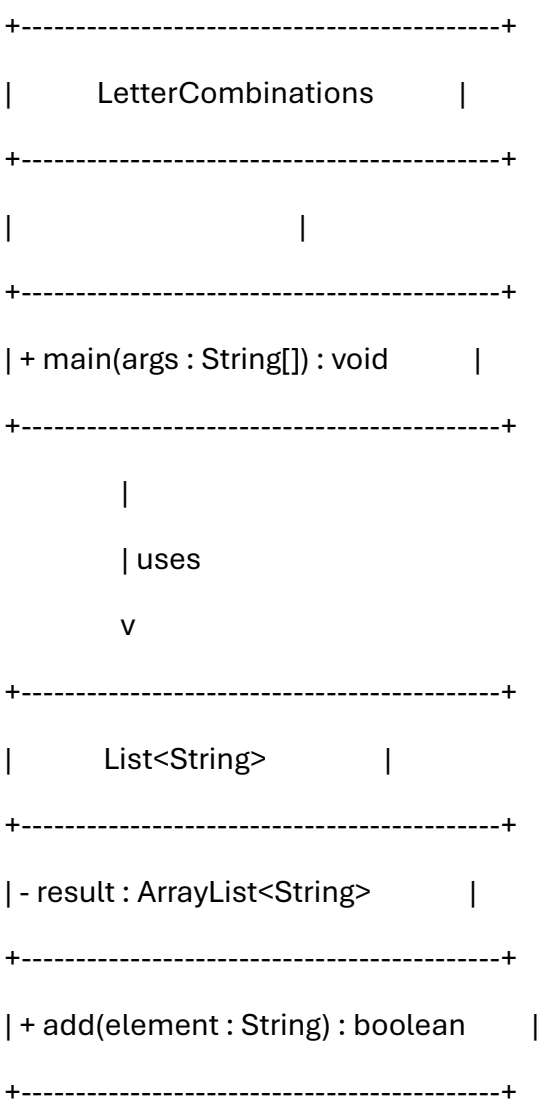
- Operating System: Windows, Linux, or macOS
- Java Development Kit (JDK): JDK 8 or above
- IDE: Eclipse, IntelliJ IDEA, NetBeans, or Visual Studio Code
- Command Prompt or Terminal

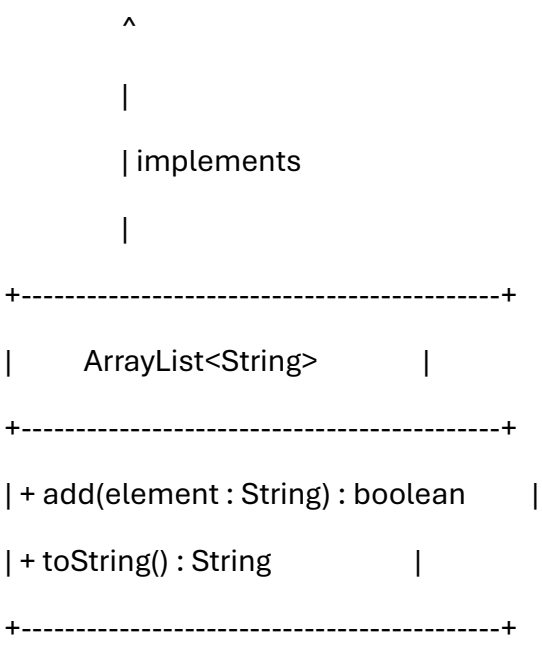
3. Algorithm / Logic Description

1. Start the program.
2. Import the java.util package to use Scanner, List, and ArrayList.
3. Create a class named LetterCombinations.
4. Create a Scanner object to accept input from the user.
5. Define a string array named phone to store the letters corresponding to each digit.
6. Read the digit string from the user.
7. Create an ArrayList named result to store the letter combinations.
8. Initially, add an empty string to the result list.
9. Read each digit from the input string using a loop.
10. Convert the character into its corresponding integer value.

11. Check whether the digit is between 2 and 9. If not, display an error message and stop the program.
12. Create a temporary ArrayList named temp.
13. Use nested loops to combine the existing combinations with the letters corresponding to the current digit.
14. Add each new combination to the temporary list.
15. Assign the temporary list to the result list.
16. Repeat the process for all input digits.
17. Display all possible letter combinations.
18. Close the Scanner object.
19. Stop the program.

4. UML Class Diagram





Description:

- LetterCombinations is the main user-defined class.
- main() is the main method where the program execution starts.
- List is an interface used to store multiple letter combinations.
- ArrayList is a class that implements the List interface.
- result stores the generated letter combinations.
- temp temporarily stores the new combinations.
- Scanner is used to accept input from the user.
- No inheritance or constructor is explicitly defined in the user-defined class.
- The program demonstrates the List interface and ArrayList from the Java Collection Framework.

5. Source Code

```
import java.util.*;


public class LetterCombinations {


   public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);

String[] phone = {
   "", "", "abc", "def", "ghi",
   "jkl", "mno", "pqrs", "tuv", "wxyz"
};

System.out.print("Enter the digits: ");
String digits = sc.nextLine();

List<String> result = new ArrayList<>();
result.add("");

for (int i = 0; i < digits.length(); i++) {

   int num = digits.charAt(i) - '0';

   if (num < 2 || num > 9) {
      System.out.println("Enter digits from 2 to 9 only.");
      sc.close();
      return;
   }

   List<String> temp = new ArrayList<>();

   for (String s : result) {
      for (char ch : phone[num].toCharArray()) {
         temp.add(s + ch);
```

```
            }
        }

        result = temp;
    }

    System.out.println("Letter Combinations: " + result);

    sc.close();
  }
}
```

6. Compilation and Execution Commands

Step 1: Save the program

Save the Java program with the filename LetterCombinations.java.

Step 2: Open Command Prompt or Terminal

Navigate to the folder where the Java program is saved.

Step 3: Compile the program

javac LetterCombinations.java

Step 4: Execute the program

java LetterCombinations

7. Output Screen

Sample Output:

Enter the digits: 23

Letter Combinations: [ad, ae, af, bd, be, bf, cd, ce, cf]

Explanation of Output:

The phone keypad contains the following letter mappings:

Digit	Corresponding Letters

2 abc

3 def

4 ghi

5 jkl

6 mno

7 pqrs

8 tuv

9 wxyz

For the input 23:

- Digit 2 represents a, b, c.
- Digit 3 represents d, e, f.
- The program combines each letter of digit 2 with each letter of digit 3.

The possible combinations are:

ad ae af

bd be bf

cd ce cf

Total number of combinations:

3 × 3 = 9

Therefore, the final output is:

[ad, ae, af, bd, be, bf, cd, ce, cf]

Important Concepts Used:

1. **Collection Framework:** Used to store and manage multiple letter combinations.
2. **List Interface:** Stores elements in an ordered collection.
3. **ArrayList:** Implements the List interface and stores the generated combinations.
4. **Scanner:** Accepts input from the user.
5. **String Array:** Stores the phone keypad letter mappings.
6. **Nested Loops:** Generate all possible letter combinations.

7. **toCharArray():** Converts a string into a character array.
8. **add():** Adds new combinations to the ArrayList.

Note: This program accepts digits from 2 to 9 only. The digit 1 and 0 do not have letter mappings in this program. The Set and Queue interfaces are not required for this particular problem.

8. Result

Thus, the Java program to generate all possible letter combinations of a phone number using the Collection Framework was successfully implemented and executed. The experiment demonstrates the List interface, ArrayList, Scanner, string handling, nested loops, and collection operations to generate and store all possible combinations.